

62. Une méthode : la fonction qui connaît son objet

``troupe[i].avancer()`` plutôt que ``avancer(i)``.

1. Ce qu'il faut comprendre

Une méthode s'écrit dans la `struct`, et travaille sur l'objet devant le point. Dans son corps, les champs s'écrivent sous leur nom nu : `x++`, et non `this->x++`.

Ce qu'elle coûte, exactement : l'adresse de l'objet est rangée dans **deux octets réservés à cette méthode**, puis c'est le même `call` qu'ailleurs. Pas de table de fonctions, pas de pile d'objets — et le compilateur compte ces deux octets comme les autres.

Ce qui reste refusé, et pourquoi : `class` (le message propose `struct`), `virtual` (il faudrait une table de fonctions et un appel indirect), l'héritage (il déplacerait les décalages des champs), les constructeurs (du code implicite à la déclaration, alors que les globales prennent leur valeur avant `main`). Et une méthode **ne prend pas d'argument** : elle reçoit déjà son objet.

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1595 octets** de cartouche.

1. UN ENREGISTREMENT — ENNEMI

```
struct Ennemi {
    uint8_t x, y, vie;

    void avancer() { x++; }
    uint8_t vivant() { return vie > 0; }
};
```

2. UN TABLEAU — TROUPE

```
Ennemi troupe[3];
```

3. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

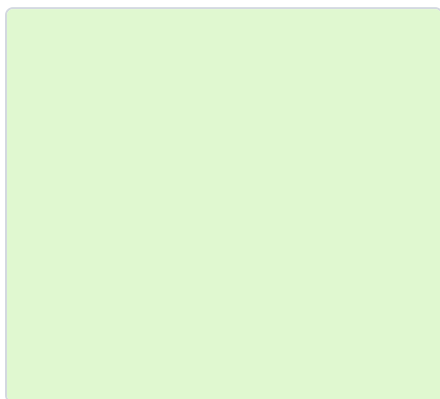
```
int main() {
    for (uint8_t i = 0; i < 3; i++) { troupe[i].x = 20 + i * 40; troupe[i].vie = 3; }
    troupe[1].vie = 0;

    while (true) {
        for (uint8_t i = 0; i < 3; i++)
            if (troupe[i].vivant()) troupe[i].avancer();

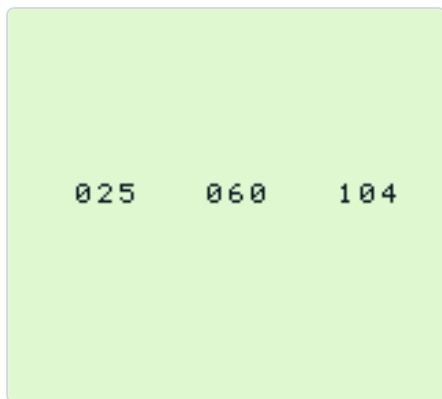
        for (uint8_t i = 0; i < 3; i++) nombre(3 + i * 6, 8, troupe[i].x);
        image();
    }
}
```

3. Ce qui se passe, étape par étape

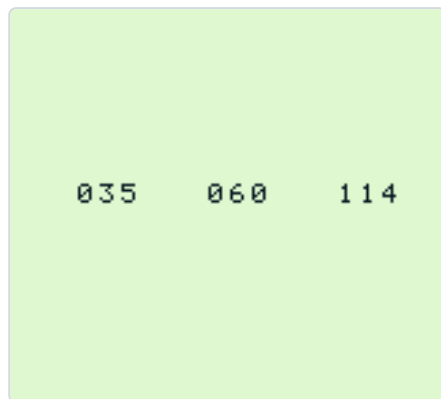
La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



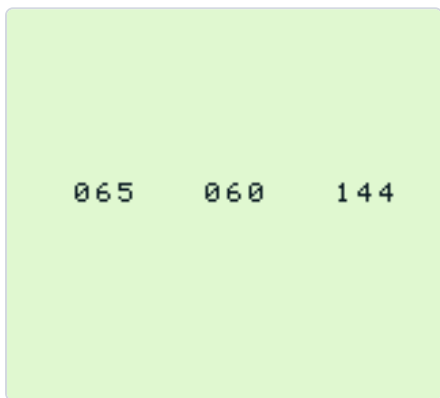
la toute première image — la console vient de démarrer — image 1



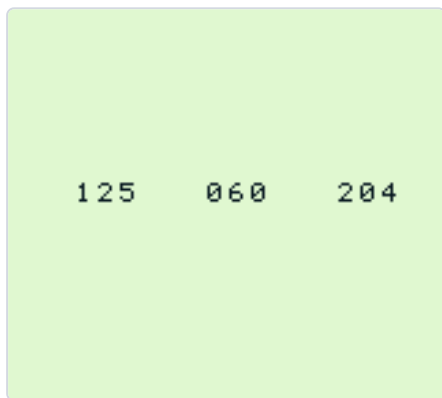
un tiers de seconde plus tard : le premier dessin est posé — image 20



au bout d'une seconde — image 60



au bout de trois secondes — image 180



au bout de sept secondes — image 420

4. Ce qu'on doit voir

Deux nombres qui montent, et celui du milieu immobile.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ le premier avance (025 → 035)
- ✓ le MORT du milieu ne bouge pas (060)
- ✓ le troisième avance (104 → 114)
- ✓ la méthode a donc écrit dans le BON objet

Ensuite : **63. Les trois façons de faire la même chose** — niveau 8 sur 10. · gameboy3 — écrire une cartouche Game Boy en C++